Arrays

22 March 2026
20:03

Arrays

It is a **Linear data structure.**

An array in PowerShell is a data structure used to store a collection of **one or more items in a single variable.** Items can be of the **same type or different types, including strings, integers, objects, or even other arrays**

The starting index starts from 0

	100	101	102	103	104	105
index	0	1	2	3	4	5

So here the **starting Array Index = 0**

Length = 6 (6 elements) also called as count

Last index = length -1

Fixed Size Array

The size or length of a standard PowerShell array is determined when it is created and cannot be changed after allocation.

$arr = @("A", "B") - here the length is fixed in the memory as per the elements inside array so called as **Fixed Size Array**

$arr[0]

$arr[1]

```
PS C:\Users\lenovo> $arr = @("A","B")
PS C:\Users\lenovo> $arr[0]
A
PS C:\Users\lenovo> $arr[0,1]
A
B
```

Here if we use **add() or remove()** methods it will throw an error.

We can configure it by using **+=** operator.

PowerShell actually creates an entirely new array with the original values plus the new item, and then copies all elements over. This process is computationally expensive and can cause significant performance issues if done repeatedly, especially in loops with large arrays

```
PS C:\Users\lenovo> $arr +="D"
PS C:\Users\lenovo> $arr.add("E")
Exception calling "Add" with "1" argument(s): "Collection was of a fixed size."
At line:1 char:1
+ $arr.add("E")
+ ~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : NotSupportedException

PS C:\Users\lenovo> $arr
A
B
C
D
```

Dynamic Size Array

A dynamic array is a variable-size list data structure that allows elements to be added or removed efficiently at runtime.

Properties of Array

Here to find the length and count we use this properties

$arr.count

$arr.length

Almost both the property returns the same value but length is Primarily used by the underlying System.Array class and Count is Implemented on most collections and PowerShell objects

```
PS C:\Users\lenovo> $arr.count
4
PS C:\Users\lenovo> $arr.length
4
```

To Get ALL property of array as we use - **get-member -inputobject $arr**

```
PS C:\Users\lenovo> get-member -InputObject $arr


   TypeName: System.Object[]

Name           MemberType            Definition
----           ----------            ----------
Count          AliasProperty         Count = Length
Add            Method                int IList.Add(Syste
Address        Method                System.Object&, msc
Clear          Method                void IList.Clear()
Clone          Method                System.Object Clone
CompareTo      Method                int IStructuralCompa
Contains       Method                bool IList.Contains
CopyTo         Method                void CopyTo(array a
Equals         Method                bool Equals(System.
Get            Method                System.Object Get(i
GetEnumerator  Method                System.Collections.
GetHashCode    Method                int GetHashCode(),
GetLength      Method                int GetLength(int d
```

Here we get multiple property that can be used with array

To check the array is of fixed size as a property

$arr.IsFixedSize

```
PS C:\Users\lenovo> $arr.Isfixedsize
True
PS C:\Users\lenovo> $rr.count
0
```

To check the array is of rank as a method

$arr.gettype()

```
PS C:\Users\lenovo> $arr.rank
1
PS C:\Users\lenovo> $arr.gettype()

IsPublic IsSerial Name                                     BaseType
-------- -------- ----                                     --------
True     True     Object[]                                 System.Array
```

$arr.indexof()

```
PS C:\Users\lenovo> $arr.indexof("R")
-1
PS C:\Users\lenovo> $arr.indexof("D")
3
```

ARRAY List

$arrlist = new-object -typename system.collection.arraylist
Get-member -inputobject $arrlist
The arrlist is present in system.collection.arraylist

```
PS C:\Users\lenovo> $arrlist = new-object -typename System.Collections.Arraylist
PS C:\Users\lenovo> get-member -inputobject $arrlist


   TypeName: System.Collections.ArrayList

Name           MemberType            Definition
----           ----------            ----------
Add            Method                int Add(System.Object value), int IList.Add(System.Object v
AddRange       Method                void AddRange(System.Collections.ICollection c)
BinarySearch   Method                int BinarySearch(int index, int count, System.Object value,
Clear          Method                void Clear(), void IList.Clear()
Clone          Method                System.Object Clone(), System.Object ICloneable.Clone()
Contains       Method                bool Contains(System.Object item), bool IList.Contains(Syst
CopyTo         Method                void CopyTo(array array), void CopyTo(array array, int arra
Equals         Method                bool Equals(System.Object obj)
GetEnumerator  Method                System.Collections.IEnumerator GetEnumerator(), System.Coll
GetHashCode    Method                int GetHashCode()
GetRange       Method                System.Collections.ArrayList GetRange(int index, int count)
GetType        Method                type GetType()
IndexOf        Method                int IndexOf(System.Object value), int IndexOf(System.Object
Insert         Method                void Insert(int index, System.Object value), void IList.Ins
InsertRange    Method                void InsertRange(int index, System.Collections.ICollection
LastIndexOf    Method                int LastIndexOf(System.Object value), int LastIndexOf(Syste
Remove         Method                void Remove(System.Object obj), void IList.Remove(System.Ob
RemoveAt       Method                void RemoveAt(int index), void IList.RemoveAt(int index)
RemoveRange    Method                void RemoveRange(int index, int count)
Reverse        Method                void Reverse(), void Reverse(int index, int count)
SetRange       Method                void SetRange(int index, System.Collections.ICollection c)
Sort           Method                void Sort(), void Sort(System.Collections.IComparer compare
ToArray        Method                System.Object[] ToArray(), array ToArray(type type)
ToString       Method                string ToString()
TrimToSize     Method                void TrimToSize()
Item           ParameterizedProperty System.Object Item(int index) {get;set;}
Capacity       Property              int Capacity {get;set;}
Count          Property              int Count {get;}
IsFixedSize    Property              bool IsFixedSize {get;}
IsReadOnly     Property              bool IsReadOnly {get;}
IsSynchronized Property              bool IsSynchronized {get;}
```

To check we can add on arrlist we use fixedsize property
$arrlist.Isfixedsize
$arrList.Add("First")
$arrList.Add("Second")

```
PS C:\Users\lenovo> $arrlist.isfixedsize
False
PS C:\Users\lenovo> $arrlist.Add("first")
0
PS C:\Users\lenovo> $arrlist.Add("Second")
1
```

It is used to hide the index which is displayed every time when we use this command
$arrList.Contains("third") | out-Null
$arrlist[0] - provide index
$arrlist[-1] - provide index
To clear all elements in the list
$arrList.clear()
To check : **$arrlist**

To change the existing values in array we use the command

$a[0] ="new value"